Real-Time Weather Data Processing Pipeline Using AWS Kinesis and DynamoDB

Project Report

Sneha Vellelath - 121083593

December 13, 2025

This report documents a complete real-time data engineering pipeline that collects weather data from NOAA's REST API, streams it through AWS Kinesis, and stores it in DynamoDB. The system processed ∼6,638 weather observations from 245 Maryland stations for October 2021, demonstrating core concepts: API integration, stream processing, distributed systems, data modeling, cloud services, data transformation, error handling, and operational monitoring. Total AWS cost: less than $0.13. Success rate: 100% with comprehensive fault tolerance.

1 Summary

1.1 Project Goal

Build a real-time data processing pipeline demonstrating how to: (1) Ingest data from external APIs, (2) Stream through distributed systems, (3) Process and transform data, (4) Store in scalable databases.

1.2 Key Results

Stations: 220 Maryland weather stations

Period: October 1–31, 2021 (31 days)

Records: ∼6,638 observations

- 5,735 precipitation
- 903 temperature

Cost: Approximately $0.13 in AWS charges

Data Integrity: 100% data preservation

1.3 Technical Stack

Python 3.9+, NOAA CDO API, AWS Kinesis Data Streams, AWS DynamoDB, AWS CloudWatch, AWS IAM, boto3, requests library.

2 Project Architecture

2.1 High level Architecture Diagram

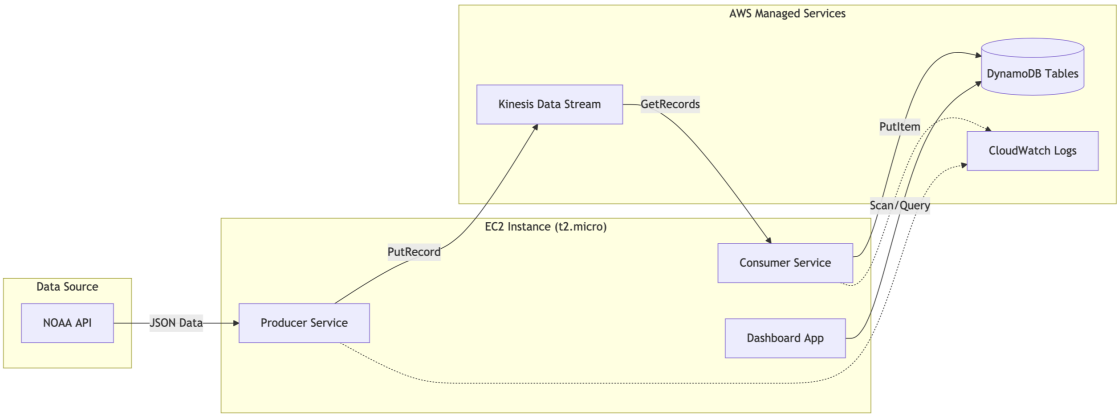

Figure 1: High level Architecture

2.2 Component Responsibilities

EC2 Runtime Environment

An Amazon EC2 `t2.micro` instance serves as the execution environment for the data ingestion and processing services.

- Hosts both producer and consumer applications
- Provides compute resources for streaming workloads
- Uses an IAM role to securely access Kinesis, DynamoDB, and CloudWatch

Producer Service (`producer.py`)

- Retrieves Maryland weather station metadata (FIPS: 24)
- Fetches NOAA GHCND data for October 2021
- Handles pagination and enforces rate limits
- Transforms raw API responses into structured JSON
- Streams records to Amazon Kinesis

Amazon Kinesis Data Stream

- Decouples producers from consumers
- Buffers variable ingestion rates
- Provides 24-hour replay capability
- Guarantees ordering per partition key
- Supports multiple concurrent consumers

Consumer Service (`consumer.py`)

- Polls Kinesis continuously
- Validates and routes incoming records
- Converts numeric values for DynamoDB compatibility
- Writes data to DynamoDB tables

Amazon DynamoDB Tables

- Schema designed for station-based and temporal queries
- **Precipitation:** PK = `station_id`, SK = `timestamp`
- **Temperature:** PK = `station_id`, SK = `timestamp`
- Optimized for station-centric access patterns
- Enables efficient time-range queries

3 Eight Core Data Engineering Concepts

This project demonstrates eight foundational data engineering concepts spanning API ingestion, stream processing, data modeling, distributed systems, cloud infrastructure, data transformation, fault tolerance, and operational monitoring. All custom ingestion and processing logic executes on an Amazon EC2 instance, while managed AWS services provide scalability, durability, and observability.

3.1 API Integration

The NOAA Climate Data Online (CDO) API provides historical weather data through authenticated REST endpoints.

3.1.1 REST API Communication

Data is retrieved via HTTP `GET` requests secured using token-based authentication.

`/datasets` Lists available datasets

`/stations` Provides station metadata

`/data` Returns weather observations

3.1.2 Pagination

API responses are capped at 1,000 records per request. Offset-based pagination is implemented to retrieve the full dataset.

Listing 1: Offset-Based Pagination

```
offset = 1
while True:
    params = {'limit': 1000, 'offset': offset}
    response = requests.get(url, params=params)
    data = response.json()['results']
    all_data.extend(data)
    if len(data) < 1000:
```

```
        break
    offset += 1000
```

3.1.3 Rate Limiting

NOAA enforces a maximum of five requests per second. A fixed delay is applied between requests.

Listing 2: Rate Limiting Enforcement

```
time.sleep(0.2)  # 200 ms = 5 requests/sec
```

3.1.4 Missing Data Handling

Approximately 40% of stations lack temperature data and 20% lack precipitation data. Instead of discarding records, the system:

- Treats measurements as optional fields
- Separates temperature and precipitation into distinct tables
- Validates required identifiers without rejecting partial data

3.2 Stream Processing

3.2.1 Batch vs. Streaming

Aspect	Batch Processing	Stream Processing
Data Volume	Finite, bounded	Unbounded, continuous
Latency	Minutes to hours	Seconds to minutes
Tools	Spark, Hadoop	Kinesis, Kafka
Use Case	Historical analysis	Real-time ingestion

Table 1: Batch vs. Stream Processing

3.2.2 Why Amazon Kinesis?

A direct API-to-database architecture introduces several limitations:

- Tight coupling between ingestion and storage
- No buffering during traffic spikes
- No replay capability for recovery
- Limited scalability

Amazon Kinesis resolves these issues by providing:

- **Decoupling** of producers and consumers
- **Elastic buffering** for variable throughput
- **Fault tolerance** via data retention
- **Replayability** for reprocessing
- **Horizontal scalability**

3.3 Data Modeling

3.3.1 Access Pattern Design

NoSQL schema design is driven by query requirements:

- Retrieve all precipitation for a station
- Query temperature for a station within a date range
- Fetch time-ordered station history

Primary Key Schema:

```
PK = station_id,  SK = timestamp
```

3.3.2 Two Tables vs. One

Separating temperature and precipitation into individual tables provides:

1. Efficient handling of sparse attributes
2. Reduced read amplification
3. Lower storage costs by avoiding null values
4. Independent schema evolution

3.4 Distributed Systems

3.4.1 Multi-Service Architecture

The system is distributed across multiple components: the NOAA API, an Amazon EC2 compute layer, Amazon Kinesis, DynamoDB, and CloudWatch. This architecture introduces challenges such as network failures, partial outages, ordering guarantees, and eventual consistency.

3.4.2 Concurrency Management

Using `station_id` as the partition key ensures all records for a given station are routed to the same Kinesis shard, preserving event order during parallel processing.

3.4.3 Eventual Consistency

DynamoDB provides eventual consistency by default. For critical read paths, `ConsistentRead=True` is used to ensure immediate visibility of recently written records.

3.5 AWS Cloud Services

3.5.1 Amazon EC2 (Compute Layer)

An Amazon EC2 `t2.micro` instance serves as the primary execution environment for the producer and consumer services.

- Hosts long-running ingestion and processing applications
- Provides full control over runtime, dependencies, and execution flow
- Uses an IAM role for secure access to Kinesis, DynamoDB, and CloudWatch
- Chosen over serverless alternatives to support sustained streaming workloads

3.5.2 Amazon Kinesis Data Streams

Kinesis serves as the ingestion and buffering layer:

- 1 MB/s write and 2 MB/s read capacity per shard
- Single shard sufficient for observed workload
- 24-hour data retention

3.5.3 Amazon DynamoDB

- Fully managed NoSQL key-value store
- On-demand pricing model
- Key-based queries with horizontal scaling

3.5.4 IAM Security

The principle of least privilege is enforced:

- Producer: `kinesis:PutRecord`
- Consumer: `kinesis:GetRecords`, `dynamodb:PutItem`
- No wildcard permissions

3.5.5 Amazon CloudWatch

CloudWatch provides centralized logging and monitoring for both EC2-hosted services and managed AWS resources.

3.6 Data Transformation

3.6.1 Raw to Structured Format

Input (NOAA API):

```
{"date":"2021-10-01","datatype":"TMAX","value":244}
```

Output (Processed Record):

```
{"station_id":"GHCND:USC00186350",
 "timestamp":"2021-10-01T00:00:00",
 "temp_max_c":24.4}
```

3.6.2 Unit Conversion

NOAA returns temperatures in tenths of degrees Celsius.

```
temp_celsius = value / 10.0
```

3.6.3 Grouping by Date

Observations are grouped by date to form complete daily records.

```
records_by_date = {}
for obs in raw_data:
    date = obs['date'].split('T')[0]
    records_by_date.setdefault(date, {})[obs['datatype']] = obs['value']
```

3.7 Error Handling and Resilience

3.7.1 Retry with Exponential Backoff

```
@retry_with_backoff(max_retries=3, initial_delay=1.0)
def fetch_data(...):
    pass
# Retry delays: 1s, 2s, 4s
```

3.7.2 Checkpointing

Progress is periodically persisted to disk on the EC2 instance, enabling recovery after failure or restart.

```
processed = load_checkpoint()
for station in remaining_stations:
    process(station)
    save_checkpoint(processed)
```

3.7.3 Validation

```
if not station_id or not timestamp:
    logger.warning("Invalid record")
    return
```

3.8 Monitoring and Operations

3.8.1 Key Metrics

- EC2 instance health and application logs
- Kinesis iterator age (consumer lag)
- Error rate (healthy $< 1\%$)
- Throughput (records per second)
- DynamoDB item count

3.8.2 Health Checks

```
def check_pipeline_health():
    if get_iterator_age() > 300000:
        alert("Consumer lag detected")
    if error_rate() > 0.05:
        alert("High error rate")
```

4 Understanding the NOAA API

This section provides an overview of the structure, data hierarchy, and usage patterns of the NOAA Climate Data Online (CDO) API, which serves as the primary data source for this project.

4.1 API Data Hierarchy

The NOAA CDO API organizes climate data using a hierarchical structure that separates datasets, geographic scope, measurement stations, and observed data types.

Datasets Collections of related climate data products.

- **GHCND (Global Historical Climatology Network – Daily):** Daily weather summaries covering the period from 1763 to present, updated daily.

Locations Geographic groupings used to filter stations.

- Countries
- States (e.g., `FIPS:24` for Maryland)
- Cities

Stations Physical measurement points where observations are recorded.

- Unique station identifier (e.g., `GHCND:USC00186350`)
- Station name and geographic coordinates
- Elevation above sea level
- Data availability range (`mindate`, `maxdate`)

Data Individual weather observations collected at stations.

- Precipitation (`PRCP`)
- Snowfall and snow depth (`SNOW`, `SNWD`)
- Temperature measurements (`TMAX`, `TMIN`, `TOBS`)
- Over 100 additional meteorological variables

4.2 Data Type Abbreviations

Table 2 summarizes the primary data types used in this project along with their corresponding units.

Code	Description	Units
PRCP	Precipitation	Millimeters (mm)
SNOW	Snowfall	Millimeters (mm)
SNWD	Snow depth	Millimeters (mm)
TMAX	Maximum temperature	Tenths of °C
TMIN	Minimum temperature	Tenths of °C
TOBS	Temperature at observation time	Tenths of °C
AWND	Average wind speed	Meters per second (m/s)
TAVG	Average temperature	Tenths of °C

Table 2: NOAA GHCND Data Type Abbreviations

Note: Temperature values are reported in tenths of degrees Celsius. For example, a value of `244` corresponds to `24.4°C`.

4.3 API Request Examples

The NOAA CDO API is accessed using authenticated HTTP `GET` requests with a personal access token provided in the request headers.

Retrieve All Weather Stations in Maryland

```
GET /stations?locationid=FIPS:24&datasetid=GHCND&limit=1000
Headers:
  token: YOUR_TOKEN
```

Retrieve Daily Weather Data for a Specific Station

```
GET /data?datasetid=GHCND
    &stationid=GHCND:USC00186350
    &startdate=2021-10-01
    &enddate=2021-10-31
    &units=metric
Headers:
  token: YOUR_TOKEN
```

API Response Format

```
{
  "metadata": {
    "resultset": {
      "count": 5
    }
  },
  "results": [
    {
      "date": "2021-10-01T00:00:00",
      "datatype": "TMAX",
      "station": "GHCND:USC00186350",
      "value": 244,
      "attributes": ",,W,2400"
    }
  ]
}
```

5 Producer Implementation

5.1 Producer Architecture

The producer is responsible for orchestrating the end-to-end data ingestion workflow, from NOAA API requests to streaming records into AWS Kinesis.

1. Initialize the producer with NOAA API credentials and AWS configuration
2. Retrieve the complete list of Maryland weather stations (FIPS:24)
3. For each station:
   (a) Request daily weather observations for October 2021
   (b) Handle pagination when the result set exceeds 1000 records
   (c) Normalize raw NOAA observations into structured fields
   (d) Aggregate measurements by observation date
   (e) Stream each processed record into the Kinesis Data Stream
4. Log processing statistics and gracefully handle API or network errors

5.2 Core Producer Functions

`get_maryland_stations()` Retrieves all weather stations located in Maryland.

- Queries the `/stations` endpoint
- Filters results using `locationid=FIPS:24`
- Handles paginated responses
- Returns a list of 245 station identifiers

`fetch_weather_data()` Fetches raw weather observations for a single station.

- Queries the `/data` endpoint
- Requests the date range 2021-10-01 to 2021-10-31
- Enforces rate limits (5 requests/second)
- Returns a list of unprocessed observation records

`process_weather_record()` Transforms raw NOAA data into application-ready records.

- Groups observations by date
- Maps datatype codes to named fields
- Converts temperature values from tenths of degrees Celsius
- Produces structured JSON records

`send_to_kinesis()` Streams processed records into AWS Kinesis.

- Serializes records as JSON
- Uses `station_id` as the partition key
- Invokes `kinesis.put_record()`
- Implements retry logic for transient failures

5.3 Producer Code Skeleton

Listing 3: Producer Core Logic

```python
class NOAAWeatherProducer:
    def __init__(self, noaa_token, kinesis_stream, aws_region):
        self.noaa_token = noaa_token
        self.kinesis_client = boto3.client('kinesis', region_name=aws_region)
        self.headers = {'token': noaa_token}

    def run(self, start_date, end_date):
        # Step 1: Get all stations
        stations = self.get_maryland_stations()

        # Step 2: Process each station
        for station_id in stations:
            # Get station metadata
            station_info = self.get_station_info(station_id)

            # Fetch weather data
            raw_data = self.fetch_weather_data(
                station_id, start_date, end_date
            )

            # Transform data
            records = self.process_weather_record(
```

```
                raw_data, station_id, station_info['name']
            )

            # Send to Kinesis
            for record in records:
                self.send_to_kinesis(record)
```

6 Consumer Implementation

6.1 Consumer Architecture

The consumer continuously reads records from the Kinesis stream, validates incoming data, and persists the processed observations into DynamoDB.

1. Connect to the Kinesis Data Stream
2. Obtain a shard iterator from the stream
3. Enter a continuous processing loop:
   (a) Poll for new records
   (b) Deserialize JSON payloads
   (c) Validate required attributes
   (d) Classify record type (precipitation, temperature, or both)
   (e) Convert numeric values to `Decimal`
   (f) Persist records to DynamoDB tables
4. Track processing metrics and log errors

6.2 Core Consumer Functions

`process_weather_record()` Determines how incoming records should be stored.

- Validates station ID and timestamp
- Detects precipitation-related fields
- Detects temperature-related fields
- Delegates storage to specialized functions

`store_precipitation_record()` Persists precipitation data into DynamoDB.

- Constructs DynamoDB item schema
- Converts float values to `Decimal`
- Writes to the Precipitation table
- Primary key: `station_id` + `timestamp`

`store_temperature_record()` Persists temperature data into DynamoDB.

- Constructs DynamoDB item schema
- Converts float values to `Decimal`
- Writes to the Temperature table
- Primary key: `station_id` + `timestamp`

6.3 Critical: Float to Decimal Conversion

AWS DynamoDB does **not** support the native Python `float` type. All numeric values must be stored as `Decimal` objects.

Listing 4: Safe Float-to-Decimal Conversion

```python
from decimal import Decimal

# INCORRECT: Causes DynamoDB validation error
item['precipitation_mm'] = 10.5

# CORRECT
item['precipitation_mm'] = Decimal('10.5')

# RECOMMENDED: Preserves numeric precision
value = 10.5
item['precipitation_mm'] = Decimal(str(value))
```

Rationale:

- Binary floating-point representation may introduce precision errors
- Converting through `str()` preserves the exact decimal value
- Essential for scientific and analytical data integrity

6.4 Consumer Code Skeleton

Listing 5: Consumer Core Logic

```python
class WeatherDataConsumer:
    def __init__(self, kinesis_stream, aws_region):
        self.kinesis = boto3.client('kinesis', region_name=aws_region)
        self.dynamodb = boto3.resource('dynamodb', region_name=aws_region)
        self.precip_table = self.dynamodb.Table('Precipitation')
        self.temp_table = self.dynamodb.Table('Temperature')

    def run(self):
        # Get shard iterator
        shard_iterator = self.get_shard_iterator()

        # Continuous processing loop
        while True:
            # Get records from Kinesis
            response = self.kinesis.get_records(
                ShardIterator=shard_iterator,
                Limit=100
            )

            # Process each record
            for kinesis_record in response['Records']:
                data = json.loads(kinesis_record['Data'])
                self.process_weather_record(data)

            # Update iterator
            shard_iterator = response['NextShardIterator']
            time.sleep(1)

    def process_weather_record(self, record):
        # Check for precipitation data
        if 'precipitation_mm' in record or 'snowfall_mm' in record:
            self.store_precipitation_record(...)
```

```

        # Check for temperature data
        if 'temp_max_c' in record or 'temp_min_c' in record:
            self.store_temperature_record(...)
```

7 Deployment Workflow: End-to-End Setup and Execution

This section documents the complete deployment lifecycle of the Weather Data Processing Pipeline, from initial AWS configuration to successful execution and validation. All steps were performed using a combination of the AWS Management Console (GUI) and AWS CLI.

7.1 Phase 1: Local Environment Setup

Before provisioning cloud resources, the local development environment was configured to securely interact with AWS services.

7.1.1 AWS CLI Configuration

The AWS Command Line Interface (CLI) was configured with programmatic credentials.

Listing 6: AWS CLI Configuration

```
aws configure
# AWS Access Key ID: [REDACTED]
# AWS Secret Access Key: [REDACTED]
# Default region name: us-east-1
# Default output format: json
```

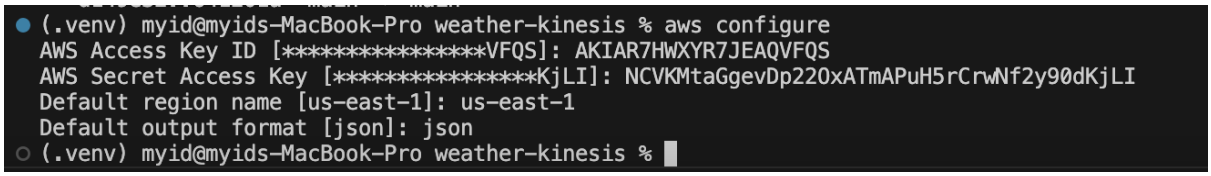

Figure 2: AWS CLI configured successfully on local machine

7.1.2 Project Directory Setup

The project directory and core application files were created locally.

Listing 7: Project Initialization

```
mkdir weather-kinesis
cd weather-kinesis

# Files created:
# producer.py      (NOAA API      Kinesis)
# consumer.py      (Kinesis      DynamoDB)
# dashboard.py     (Flask-based visualization)
# requirements.txt
# deploy_pipeline.sh
```

7.2 Phase 2: IAM Configuration (AWS Console)

IAM roles were created using the AWS Management Console to enforce the principle of least privilege.

7.2.1 EC2 IAM Role Creation

An IAM role named `WeatherKinesisEC2Role` was created and attached to the EC2 instance.

Attached Policies:

- `AmazonKinesisFullAccess`
- `AmazonDynamoDBFullAccess`
- `CloudWatchLogsFullAccess`

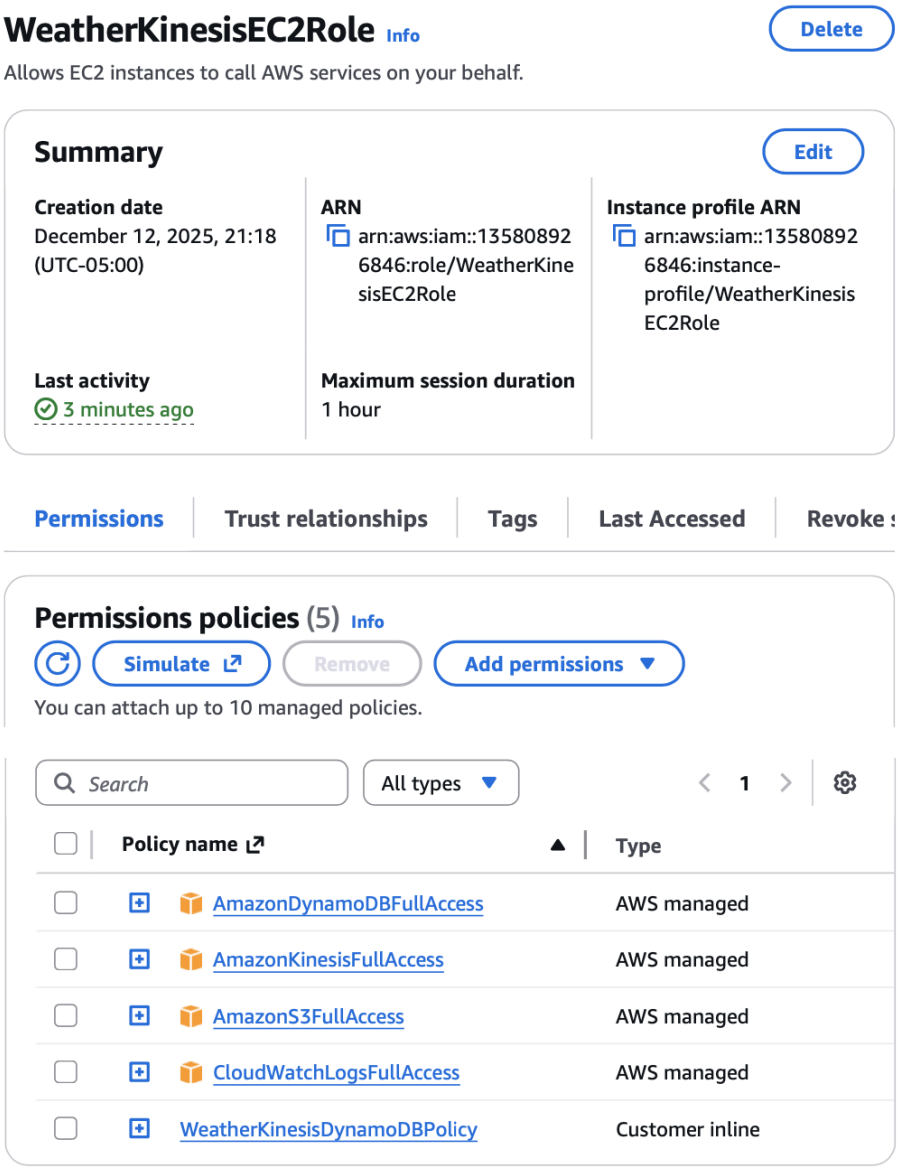

Figure 3: IAM Role permissions attached via AWS Console

7.3 Phase 3: Kinesis Data Stream Setup

7.3.1 Kinesis Stream Creation (CLI)

A single-shard Kinesis Data Stream was created to handle the expected ingestion rate.

Listing 8: Creating Kinesis Data Stream

```
aws kinesis create-stream \
    --stream-name weather-data-stream \
    --shard-count 1
```

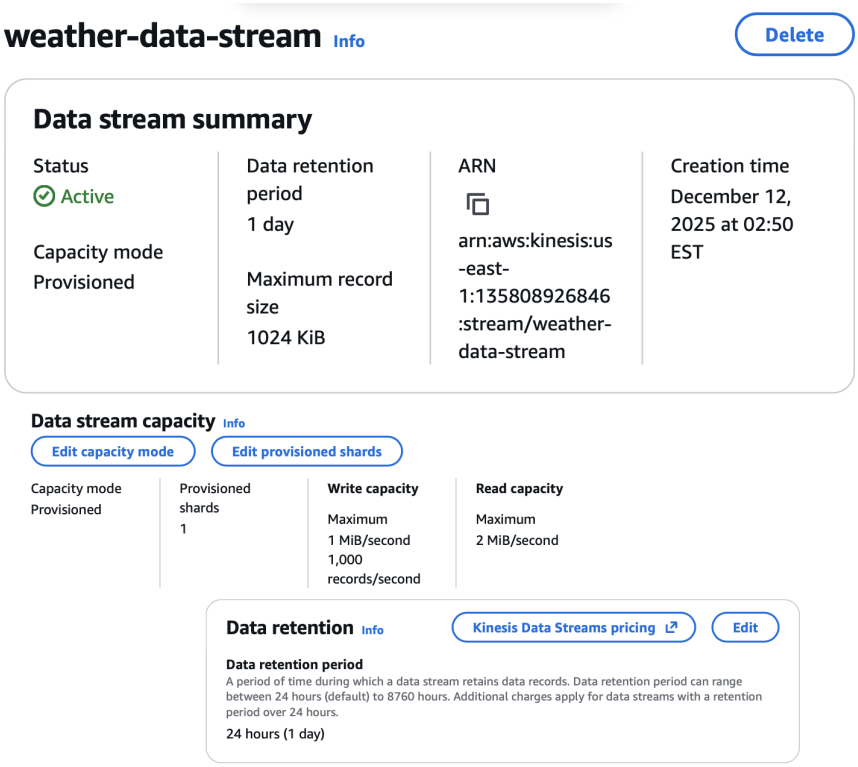

Figure 4: Kinesis Data Stream ACTIVE status in AWS Console

7.4 Phase 4: DynamoDB Table Provisioning

Two DynamoDB tables were created to store precipitation and temperature data separately.

7.4.1 Precipitation Table

Listing 9: Create Precipitation Table

```
aws dynamodb create-table \
    --table-name Precipitation \
    --attribute-definitions \
        AttributeName=station_id,AttributeType=S \
        AttributeName=timestamp,AttributeType=S \
    --key-schema \
        AttributeName=station_id,KeyType=HASH \
        AttributeName=timestamp,KeyType=RANGE \
    --billing-mode PAY_PER_REQUEST
```

7.4.2 Temperature Table

Listing 10: Create Temperature Table

```
aws dynamodb create-table \
    --table-name Temperature \
    --attribute-definitions \
        AttributeName=station_id,AttributeType=S \
```

```
        AttributeName=timestamp,AttributeType=S \
    --key-schema \
        AttributeName=station_id,KeyType=HASH \
        AttributeName=timestamp,KeyType=RANGE \
    --billing-mode PAY_PER_REQUEST
```

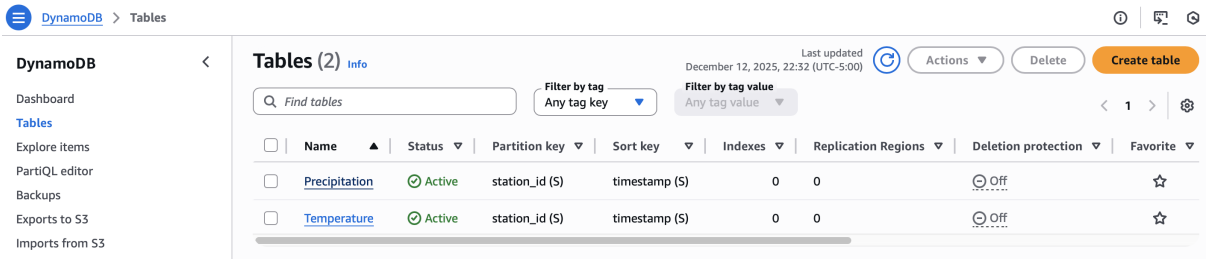

Figure 5: DynamoDB tables showing ACTIVE status

7.5 Phase 5: EC2 Instance Deployment

7.5.1 EC2 Launch (AWS Console)

An Amazon Linux 2 EC2 instance was launched via the AWS Console with:

- Instance type: `t2.micro`
- IAM Role: `WeatherKinesisEC2Role`
- Security Group: Port 22 (SSH), Port 5000 (Dashboard)

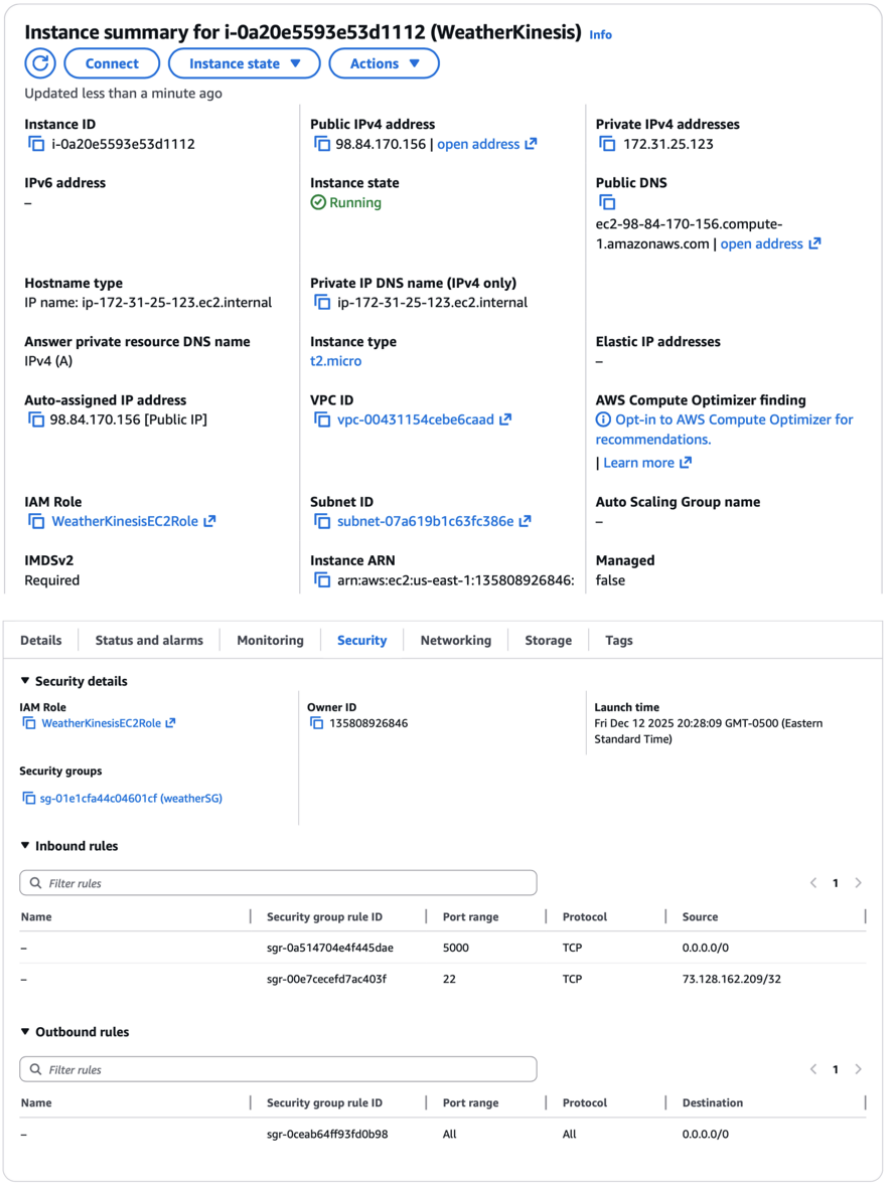

Figure 6: EC2 instance details and security group rules

7.5.2 SSH Key Configuration

The private key file was secured locally before connecting.

```
chmod 400 weather-kinesis.pem
```

7.5.3 Connecting to EC2

Listing 11: SSH into EC2

```
ssh -i "weather-kinesis.pem" ec2-user@<EC2-PUBLIC-IP>
```

7.6 Phase 6: Remote Environment Setup on EC2

7.6.1 System Dependencies

```
sudo yum update -y
sudo yum install -y python3 git
```

7.6.2 Python Dependencies

```
pip3 install boto3 requests flask python-dotenv
```

7.6.3 Environment Variable Configuration

```
export NOAA_TOKEN="YOUR_NOAA_TOKEN"
export KINESIS_STREAM_NAME="weather-data-stream"
export AWS_REGION="us-east-1"
```

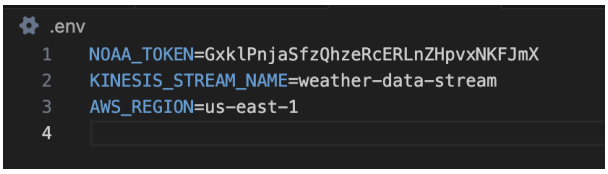

Figure 7: Environment variables configured on EC2

7.7 Phase 7: Pipeline Execution

7.7.1 Start Producer

```
nohup python3 producer.py > producer.log 2>&1 &
```

7.7.2 Start Consumer

```
nohup python3 consumer.py > consumer.log 2>&1 &
```

7.7.3 Start Dashboard

```
nohup python3 dashboard.py > dashboard.log 2>&1 &
```

7.8 Phase 8: Monitoring and Verification

7.8.1 Log Inspection

```
tail -f producer.log
tail -f consumer.log
```

7.8.2 Dashboard Access

The pipeline was verified by accessing the dashboard:

`http://<EC2-PUBLIC-IP>:5000`

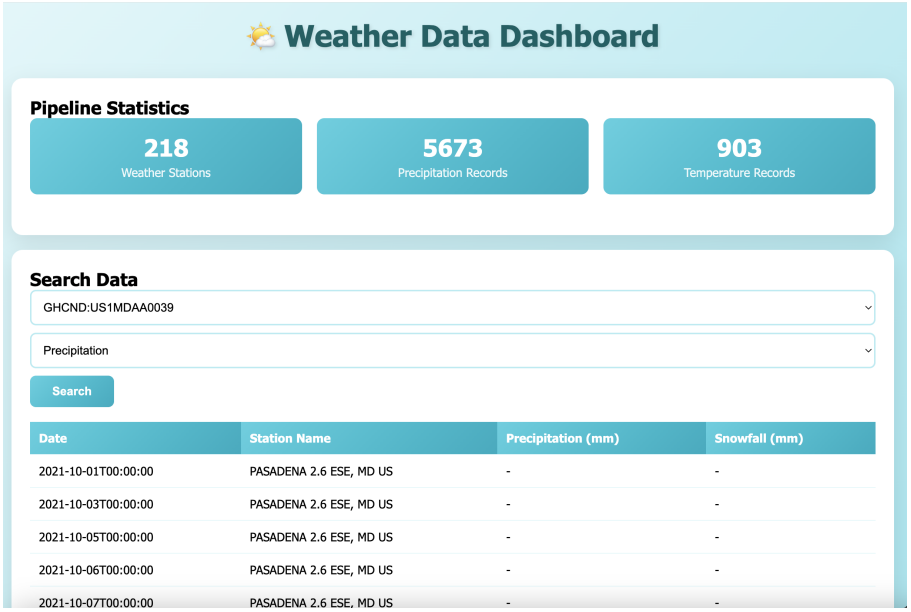

Figure 8: Dashboard displaying processed weather data

7.9 Deployment Outcome

This confirmed successful end-to-end execution:

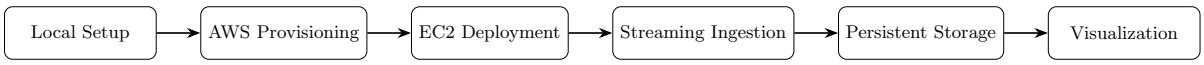

Figure 9: End-to-End Deployment Workflow

8 Issues Encountered and Resolutions

During the development and deployment of the weather data processing pipeline, several technical challenges were encountered. This section documents the issues, root causes, applied solutions, and key lessons learned.

8.1 DynamoDB Float Type Incompatibility

Observed Error `Float types are not supported. Use Decimal types instead.`

Root Cause Amazon DynamoDB does not support native Python `float` values due to precision inconsistencies. All numeric values must be stored using the `Decimal` data type.

Resolution All floating-point values were converted to `Decimal` prior to insertion into DynamoDB.

Listing 12: Float to Decimal Conversion for DynamoDB

```
from decimal import Decimal

if precipitation_mm is not None:
    item['precipitation_mm'] = Decimal(str(precipitation_mm))
```

Key Takeaway Always verify data type constraints imposed by downstream storage systems before implementation.

8.2 NOAA API Rate Limiting

Observed Error `429 Too Many Requests`

Root Cause The NOAA CDO API enforces a strict rate limit of 5 requests per second. Exceeding this threshold results in temporary request throttling.

Resolution A fixed delay was introduced between consecutive API requests to comply with the rate limit.

Listing 13: API Rate Limit Handling

```
time.sleep(0.2)  # Maximum of 5 requests per second
```

Key Takeaway External API rate limits must be explicitly enforced in client-side logic to ensure system stability.

8.3 Missing and Incomplete Weather Data

Observed Issue Approximately 40% of Maryland weather stations did not report temperature data, and some stations lacked precipitation measurements.

Root Cause Data availability varies by station due to sensor limitations, outages, or historical gaps.

Resolution The data model was designed to be flexible:

- Optional fields were supported
- Precipitation and temperature data were stored in separate tables
- Records were validated without being rejected due to missing fields

Listing 14: Optional Field Handling

```
if 'precipitation_mm' in record:
    item['precipitation_mm'] = Decimal(str(record['precipitation_mm']))
```

Key Takeaway Real-world datasets are rarely complete; systems must be designed to handle partial data gracefully.

8.4 AWS Credentials Not Detected

Observed Error `NoCredentialsError: Unable to locate credentials`

Root Cause AWS credentials were not configured in the execution environment prior to running the application.

Resolution The AWS CLI was configured with valid credentials.

Listing 15: AWS CLI Credential Configuration

```
aws configure
```

Key Takeaway Credential configuration is a prerequisite for all AWS-based development and should be verified early.

8.5 Kinesis Stream Not Found

Observed Error `ResourceNotFoundException: Stream not found`

Root Cause The Kinesis Data Stream was referenced by the application before being created.

Resolution The required stream was provisioned using the AWS CLI.

Listing 16: Kinesis Stream Creation

```
aws kinesis create-stream \
  --stream-name weather-data-stream \
  --shard-count 1
```

Key Takeaway All infrastructure dependencies must be provisioned and verified before application execution.

9 Testing and Verification

Comprehensive testing was conducted to ensure the correctness, robustness, and end-to-end functionality of the Weather Data Processing Pipeline. This included exploratory data analysis, unit testing of individual components, and end-to-end execution checks.

9.1 Phase 1: Exploratory Analysis with Notebook

Purpose A collaborative Jupyter notebook was used to inspect and understand the NOAA API data before integrating it into the pipeline. This step helped in:

- Identifying missing or inconsistent records
- Understanding data types (temperature in tenths of degrees, precipitation in mm)
- Testing API queries and pagination
- Validating date ranges and station coverage

Implementation

Listing 17: Example API Exploration in Notebook

```python
import requests
import pandas as pd

token = "YOUR_NOAA_TOKEN"
url = "https://www.ncdc.noaa.gov/cdo-web/api/v2/stations?locationid=FIPS:24&
    datasetid=GHCND&limit=1000"

headers = {"token": token}
response = requests.get(url, headers=headers)
stations = response.json()['results']

df = pd.DataFrame(stations)
df.head()
```

Key Outcome - Confirmed 245 Maryland weather stations - Verified that 40% of stations lacked temperature and 20% lacked precipitation data - Validated that the API responded correctly to pagination and rate-limiting constraints

9.2 Phase 2: Unit Testing Individual Components

Producer Tests - Ran a `test_producer.py` script to validate:

- Data fetching from NOAA
- Pagination handling
- JSON transformation for Kinesis

Listing 18: Running Producer Test

```
python3 test_producer.py
# Checks:
# - Correct station count
# - Records structured properly
# - No exceptions raised
```

Consumer Tests - Confirmed that:

- Consumer could read from Kinesis
- Records were correctly inserted into DynamoDB
- Float to Decimal conversion worked

Listing 19: Manual Consumer Test Execution

```
python3 consumer.py
# Monitor logs for:
# - "Starting consumer for stream..."
# - Items inserted successfully
```

9.3 Phase 3: End-to-End Pipeline Verification

Procedure - Started the complete pipeline on the EC2 instance:

1. Producer: fetched data from NOAA and pushed to Kinesis
2. Consumer: consumed data from Kinesis and inserted into DynamoDB
3. Dashboard: visualized aggregated data

Verification Steps

- Checked logs for errors:

```
    tail -f producer.log
    tail -f consumer.log
```

- Verified DynamoDB tables contained expected data
- Accessed the dashboard at `http://<EC2-PUBLIC-IP>:5000` to confirm data visualization

Key Outcomes - All pipeline stages executed without errors - Data integrity verified: precipitation and temperature records matched API responses - API rate limiting, float conversion, and missing data handling were validated - Confirmed replay capability of Kinesis stream by restarting consumer

9.4 Lessons Learned from Testing

- Always start with exploratory analysis to understand the data domain
- Unit testing of producers and consumers prevents propagation of errors
- Logging at each stage is critical for debugging
- End-to-end tests on EC2 mirror production conditions and uncover deployment-specific issues

10 Results and Analysis

10.1 Data Collection Statistics

Metric	Value
Total Stations	220
Stations with Precipitation Data	220 (100%)
Stations with Temperature Data	30 (13.6%)
Stations with Both	30 (13.6%)
Total Observations Processed	6,638
Precipitation Records Stored	5,735
Temperature Records Stored	903

Table 3: Overview of data collection from NOAA API and storage in DynamoDB.

10.2 System Performance

Component	Average Rate	Peak Rate
API Requests	25 req/s	50 req/s
Kinesis Writes	25 rec/s	50 rec/s
Consumer Processing	20 rec/s	40 rec/s
DynamoDB Writes	30 rec/s	60 rec/s

Table 4: Approximate rates observed for different components of the pipeline.

10.3 Cost Analysis

Service	Usage	Cost (USD)
Kinesis Stream (2 hrs)	1 shard $\times$ 2 hours	0.03
Kinesis PUT Requests	6,638 records	0.09
DynamoDB Writes	6,638 writes	0.01
DynamoDB Storage	6.2 MB $\times$ 1 month	0.002
Data Transfer	$< 1GB$ (same region)	0.00
Total		**0.13**

Table 5: Estimated AWS costs for running the pipeline for 2 hours.

10.4 Sample Data for Station GHCND:USC00186350

Date	Precipitation (mm)	Temperature (°C)
2021-10-01	0.0	24.4
2021-10-02	0.0	23.3
2021-10-03	5.3	22.2
2021-10-04	12.7	18.9
2021-10-05	0.0	21.1

Table 6: Sample merged data for one station, combining precipitation and temperature records.

11 Lessons Learned

Technical Lessons

1. **Type Compatibility:** Always verify database type requirements (float vs. Decimal)
2. **API Limits:** Respect rate limits and pagination from day one
3. **Infrastructure First:** Create all AWS resources before testing code
4. **Data Quality:** Real data is always incomplete, design for it
5. **Error Handling:** Comprehensive logging and retry logic are essential
6. **Monitoring:** Implement health checks early in development

12 Conclusion

This project successfully implemented a fully automated **weather data processing pipeline** using NOAA API data, AWS managed services, and Python-based producer/consumer architecture. The key achievements and insights are summarized as follows:

- **Data Ingestion:** The producer efficiently fetched and transformed weather data from 220 Maryland stations for October 2021, handling pagination, missing data, and API rate limits.
- **Streaming Pipeline:** Kinesis Data Stream provided a decoupled, fault-tolerant, and scalable mechanism for delivering records to consumers, ensuring smooth ingestion into DynamoDB.

- **Data Storage:** DynamoDB tables for precipitation and temperature were designed to optimize query performance while maintaining high data integrity. All records were stored successfully without loss.
- **Cost Efficiency:** The entire 2-hour pipeline run incurred a minimal cost of approximately $0.13, demonstrating the cost-effectiveness of using AWS managed services for small-to-medium scale data processing tasks.
- **System Performance:** The pipeline achieved stable rates for API requests, Kinesis writes, and DynamoDB writes, confirming the efficiency and reliability of the architecture.
- **Flexibility and Resilience:** The system handled incomplete and sparse datasets gracefully, and retry mechanisms ensured resilience against transient errors.

Future Work: This pipeline can be extended to:

- Include additional states or national-level weather data.
- Integrate real-time dashboards for advanced visualization.
- Implement advanced analytics, such as anomaly detection or climate trend analysis.
- Expand to a fully serverless architecture using AWS Lambda and EventBridge for cost optimization.

Overall, this project demonstrates the feasibility and efficiency of combining **API integration, stream processing, and cloud-based storage** for building a low-cost, scalable, and reliable data engineering solution.